

Topic Voting Platform

Architectural Blueprint & Implementation Guide

Frontend:
Next.js

Backend: Go
(Fiber)

Database:
SQLite

Real-time:
WebSockets

1. Database Schema (SQLite)

A relational structure to handle dynamic voting weights and regional constraints.

- **Rooms:** `id, name, connection_key (unique), admin_id, created_at`
- **Subjects:** `id, room_id, name, image_url, description`
- **Regions:** `id, room_id, name (e.g., Region A), weight (int)`
- **Votes:** `id, subject_id, region_id, user_hash, created_at`

2. Backend Architecture (Go + Fiber)

The Hub/Client Pattern

In Go, manage real-time traffic using a Hub that maintains active connections per room. Use `gofiber/contrib/websocket` for seamless integration with the Fiber router.

```
type RoomHub struct {  
    // Registered clients in this specific room  
    clients map[*Client]bool  
    // Inbound messages to be broadcasted  
    broadcast chan []byte  
}
```

```
// Global registry of active rooms
var activeRooms = make(map[string]*RoomHub)
```

3. Implementation Workflow

Voting Logic Flow

1. **Handshake:** Client joins via `/ws/:connection_key`.
2. **Validation:** Go backend verifies the key in SQLite and upgrades the connection.
3. **Persistence:** When a vote is received, it is written to the SQLite `Votes` table.
4. **Broadcasting:** The Hub calculates new weighted totals and pushes a JSON update to all clients in that room.

4. Advanced Enhancement Ideas

Role-Based Weighted Voting

Assign different "Power Levels" to users. An admin could assign weights not just to regions, but to specific user tokens (e.g., Moderator vote = 10, Standard vote = 1).

Live Visualization

Implement a "Racing Bar" chart on the frontend. As votes stream in via WebSocket, the subjects re-order themselves dynamically based on real-time tallies.

Time-Bound Sessions

Add an expiration timestamp to rooms. The backend can automatically broadcast a "Voting Closed" signal and terminate socket connections once the deadline is reached.

5. Initial Go Fiber Socket Handler

```
app.Get("/ws/:key", websocket.New(func(c *websocket.Conn) {
    roomKey := c.Params("key")

    // 1. Verify roomKey exists in SQLite
    // 2. Register client to the Hub for this roomKey

    defer func() {
        // Unregister on disconnect
        c.Close()
    }()

    for {
        mt, msg, err := c.ReadMessage()
        if err != nil {
            break
        }
        // Handle incoming vote payload and trigger broadcast
    }
}))
```